SIMATS ENGINEERING

ASSIGNMENT 1

CAMPUS SMART-PARKING SYSTEM

Object-Oriented Java Model

NAME	DHINESH KARTHI M
REGISTER NO	192525358
COURSE CODE	CSA0912
COURSE NAME	JAVA PROGRAMMING

SCENARIO / TECHNICAL ASSIGNMENT REPORT

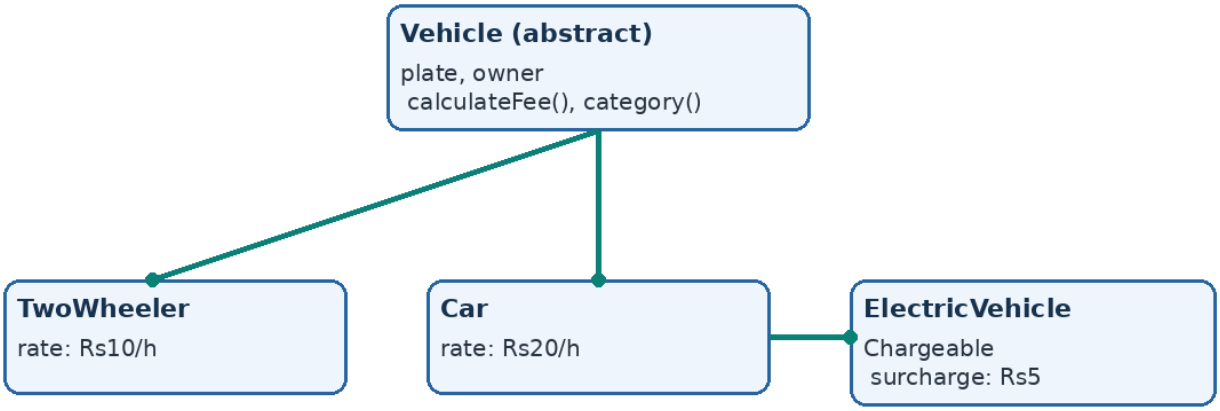
Campus Smart-Parking System

Object-Oriented Java Assignment 1 | Tasks 1–12 | Register No. 192525358

I developed this Java campus smart-parking project to apply object-oriented programming concepts.

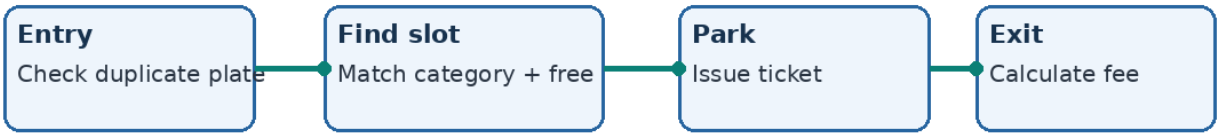
I used the stated sample tariffs and tested the listed parking scenarios.

Vehicle UML



Inheritance: ElectricVehicle extends Car; Car and TwoWheeler extend Vehicle

Parking Flowchart



Monthly pass => zero fee; otherwise hourly fee. Release the slot on valid exit.

Task 1: Problem Analysis and Object Identification

Problem: allocate compatible slots for two-wheelers, cars and EVs, record entry/exit, charge fees, and apply monthly passes. Classes: Vehicle (identity, base pricing contract), TwoWheeler, Car, ElectricVehicle, Slot (floor, number, type, occupied plate), Ticket (vehicle, slot, entry timestamp), Pass (valid-through date), ParkingFacility (allocation, departure, reporting), Main (console UI). Business rules assumed for the demonstrator: Rs10/h bike, Rs20/h car, EV Rs20/h plus Rs5 per charging-capable parking session; charge a minimum of one hour and round up partial hours; valid monthly pass waives the entire exit fee. A real specification should confirm these rules with faculty. Design comparison: parallel arrays are simple but make mismatched indexing and invalid states likely; an object collection groups attributes safely and supports extensions. The facility uses an ArrayList of Slot objects and a HashMap keyed by normalized registration number.

Task 2: Data Representation

The model uses String for plates and names, int for floor/slot/rate paise, long for minutes and fee totals, LocalDateTime for entry/exit, LocalDate for passes, boolean for availability. Money is stored as whole paise in long to avoid floating-point rounding; never cast double to int for billing. Duration.between(...).toMinutes() returns long, and checked multiplication detects overflow. A 1D List<Slot> is used: each Slot stores a floor identifier; this is flexible for differing floor capacities. A jagged Slot[][] array would also allow different floors to have different numbers of spaces but requires explicit bounds and resizing. The model never casts superclass Vehicle references to subclasses without type checks.

Task 3: Control Logic and Pseudocode

SIMATS-style pseudocode (allocation): START; INPUT vehicle, entryTime; IF plate is already active THEN DISPLAY duplicate-entry error AND STOP; FOR each slot in available slots DO IF free AND slot.type = vehicle.category THEN mark slot occupied; create ticket; save ticket by plate; increment totalEntries; RETURN ticket; END IF; END FOR; DISPLAY no slot; STOP. Fee pseudocode: START; INPUT plate, exitTime; FIND active ticket; IF missing THEN ERROR; minutes = exitTime - entryTime; IF minutes < 0 THEN ERROR; IF valid pass on exit date THEN fee=0 ELSE hours=MAX(1, CEIL(minutes/60)); fee=hours*vehicle.hourlyRate + EV charging surcharge; END IF; release slot and remove active ticket; OUTPUT fee; STOP. Code implementation: parking.service.ParkingFacility.enter(...) and exit(...). The actual starter snippet mentioned by the assessment was not provided, so this is an original working equivalent, not an exact fill-in of unseen blanks.

Sample fee: car 61 minutes -> ceil(61/60)=2 hours; 2 × Rs20 = Rs40. EV 60 minutes -> Rs20 + Rs5 = Rs25.

Task 4: Classes, Objects and Constructors

Vehicle is an abstract superclass with a parameterized constructor and copy constructor. ParkingFacility offers overloaded constructors: ParkingFacility() chains to ParkingFacility(2); ParkingFacility(int floors) configures floor count. Slot, Ticket and Pass use validating parameterized constructors. A compiler-provided default constructor is not generated when a custom constructor is declared; a no-argument constructor has been defined explicitly where useful. Constructor chaining uses this(2) in ParkingFacility and super(plate,owner) in subclasses. Conceptually local references live on stack frames while objects are allocated on the managed heap; JVM implementation optimizations may differ. Examples: new Car("TN01AA1234","Student"); new ParkingFacility(); new ParkingFacility(5).

Runnable examples: ParkingFacility() / ParkingFacility(5) demonstrate constructor overloads; FeeDemo.show(v) / FeeDemo.show(v, true) demonstrate method overloading.

Task 5: Static Members and Object Life Cycle

Ticket.issued is a private static count incremented for each ticket; ParkingFacility.totalEntries is a private static counter of entries for the running JVM. Per-facility live occupancy is active.size(), not static, because independent facilities must not share active occupancy. Class initialization occurs when the JVM first actively uses a class; objects are constructed with new, remain reachable through maps and local references, and become eligible for garbage collection after all references disappear. Garbage collection timing is controlled by the JVM and cannot be guaranteed. These counters are demonstration-level; they reset at application restart and are not thread-safe. Production use would require storage and synchronization.

Task 6: Inheritance Hierarchy and Constructor Order

Vehicle is abstract. TwoWheeler extends Vehicle; Car extends Vehicle; ElectricVehicle extends Car and implements Chargeable. Constructors execute superclass constructors before subclass bodies. Example new ElectricVehicle("EV1","Visitor"): Object constructor -> Vehicle(String,String) -> Car(String,String) -> ElectricVehicle(String,String). User roles (student/staff/visitor) are stored as owner labels in this minimal demonstrator; a dedicated User superclass and role subclasses are a proposed extension if distinct permissions or discounts are required. See UML diagram for the implemented vehicle hierarchy.

Task 7: Method Overriding and Object Contracts

TwoWheeler and Car override category() and hourlyRatePaise(). ElectricVehicle overrides category() and calculateFee(long), reusing super.calculateFee(minutes) and adding the charging surcharge. @Override requests compile-time verification that a method actually overrides a superclass or interface method. Vehicle overrides equals() and hashCode() so plates normalized to uppercase determine vehicle identity; if two vehicles are equal then their hash codes must agree. toString() gives a readable description. Plate-based equality is a design decision and assumes unique plate identifiers.

Task 8: Polymorphism

Runtime polymorphism: Vehicle v = new ElectricVehicle("EV1","Visitor"); v.calculateFee(60) dispatches to ElectricVehicle.calculateFee, returning 2500 paise. Interface checking: if (v instanceof Chargeable) { int surcharge=((Chargeable)v).chargingSurchargePaise(); }. Compile-time overload example: log(Vehicle v) and log(Vehicle v, boolean detailed) can differ by parameter list. ParkingFacility() and ParkingFacility(int floors) are constructor overloading. Java chooses overload signatures at compile-time, while overrides are dispatched at runtime. The mentioned Vehicle/FeeDemo source snippet was not included, so the supplementary FeeDemo file demonstrates these concepts independently.

Runnable examples: ParkingFacility() / ParkingFacility(5) demonstrate constructor overloads; FeeDemo.show(v) / FeeDemo.show(v, true) demonstrate method overloading.

Task 9: Abstraction

abstract class Vehicle provides shared validated identity and abstract category()/hourlyRatePaise() contracts. interface Chargeable declares chargingSurchargePaise(); ElectricVehicle implements it. interface Payable declares calculateFee(long) and can be implemented by payment-capable services; the project includes the interface as the required abstraction without falsely claiming every vehicle implements it. Optional improvement: have Vehicle implement Payable, ensuring one consistent billing contract through the interface.

Task 10: Encapsulation and Packages

Package layout: parking.model contains Vehicle, subclasses, Slot, Ticket, Pass, Chargeable, Payable; parking.service contains ParkingFacility; parking.app contains Main; tests contains ParkingTest. Fields are private/final where possible, and callers access data through getters. Constructors validate nonblank plates/owners, floor bounds, allowed slot categories and pass dates; entry rejects duplicate plates and unavailable slots; exit rejects missing vehicles and timestamps before entry. The read-only slot list prevents callers from changing list membership. For stricter encapsulation, expose only immutable snapshots instead of mutable Slot references.

Task 11: Testing and Defence

Compile: javac -d out $(find src -name '*.java') tests/ParkingTest.java. Run: java -cp out ParkingTest. Execute application: java -cp out parking.app.Main. Verified 14 checks: floor capacity=10; compatible allocation; occupancy increment; duplicate reject; bike 1 min=Rs10; occupancy decrement; car 61 min=Rs40; EV 60 min=Rs25; valid pass=Rs0; reject exit before entry; failed exit retains slot; car zero duration=Rs20 minimum; full bike section rejected; case-insensitive plate equals. Manual comparison: bike 1 minute -> ceil(1/60)=1 -> Rs10; car 61 minutes -> ceil(61/60)=2 -> Rs40; EV 60 minutes -> 1*Rs20 + Rs5 -> Rs25. Execution tested against Java 21. Notes: data resides in memory and does not persist across runs; timestamps use local time, and concurrency is not supported.

Check	Manual expectation	Java result
Bike for 1 min	Rs10	PASS
Car for 61 min	Rs40	PASS
EV for 60 min	Rs25	PASS
Active monthly pass	Rs0	PASS
Exit earlier than entry	Reject	PASS
Duplicate plate entry	Reject	PASS
Compatible section full	Reject	PASS

Task 12: Day-4 Design Challenge

E-bikes: introduce ElectricBike extends TwoWheeler implements Chargeable and configure new slot compatibility; MODIFY vehicle taxonomy and slot policy, retain facility abstraction. Peak delivery vans: add DeliveryVan extends Vehicle and a PricingPolicy strategy interface receiving arrival/exit timestamps, with an explicit peak interval; REDESIGN price calculation responsibility rather than hard-code conditional branches. One-day passes: refactor Pass into a validity interval with start/end dates and PassType; MODIFY pass classes and fee waiver logic. Five floors: current ParkingFacility(5) works with 10 slots/floor; RETAIN the list-based facility (performance is acceptable for a campus scale). Add persistence, authentication, audit logs, thread-safe allocation, EV charge session tracking, accessibility spots and security policies before production. Decisions emphasize the Open-Closed Principle and reduced coupling.

Submission checklist

Repository folders Task01–Task12; executable Java sources; 14 passing automated checks; UML and flowchart illustrations; detailed README; this report. GitHub pushes, granting faculty access, real progressive commits, and portal upload require action from the student account. The supplied overview does not contain the actual starter snippets, numerical tariffs or complete grading sheet; treat implementation assumptions accordingly.